

Mawlana Bhashani Science and Technology University
Department of Information and Communication Technology

Shashtho Record

A Medical History Tracker with Automatic Antibiotic Labelling

For the course of
Software Engineering and Project Management
ICT 3107

Project Proposal and Design Report

Submitted by: Ishtiak Ahmed Ayon
Student ID: IT24039
Session: 2023-24

Supervisor: Prof. Dr. Ziaur Rahman
Designation: Professor, Dept. of ICT, MBSTU

GitHub: https://github.com/ishtiakahmedayon/SEPM-Capstone-Project

Contents

1 Introduction **2**
1.1 Background . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 2
1.4 Scope . . . 2
1.5 Project Summary . . . 3

2 Stakeholders, Existing Systems and Feasibility **3**
2.1 Stakeholders . . . 3
2.2 Existing Approaches . . . 3
2.3 Feasibility Study . . . 4

3 Requirements **4**
3.1 Functional Requirements . . . 4
3.2 Non-Functional Requirements . . . 5
3.3 Assumptions and Constraints . . . 5

4 Software Development Model: Iterative and Incremental **5**
4.1 Overview . . . 5
4.2 Why This Model . . . 6

5 Project Schedule, Milestones and Deliverables **6**
5.1 Iteration Plan . . . 6
5.2 Milestones . . . 6
5.3 Deliverables . . . 7
5.4 Resources and Effort . . . 7
5.5 Gantt Chart . . . 8

6 UML and Design Diagrams **9**
6.1 Data Flow Diagrams . . . 9
6.2 Use Case Diagram . . . 10
6.3 Activity Diagram . . . 11
6.4 Entity Relationship Diagram . . . 12
6.5 Class Diagram . . . 13
6.6 Object Diagram . . . 14

7 Implementation **14**
7.1 Architecture . . . 14
7.2 Classification Rule . . . 15
7.3 Screens . . . 15

8 Testing Plan **16**

9 Risk Management **16**

10 Future Scope **16**

11 Conclusion **17**

1 Introduction

1.1 Background

Patients visit doctors many times over the years, yet the record of those visits is usually a bundle of paper prescriptions. Symptoms, diagnoses and medicines are forgotten or lost. Knowing which antibiotics were taken, when, and for how long is especially important: repeated or incomplete antibiotic courses contribute to antibiotic resistance, and doctors often ask for this history.

Prescriptions in Bangladesh mostly carry brand names (for example Azithro, Zimax, Ciprocin, Cef-3, Flagyl) rather than generic names. A tracker that only knows generic names cannot recognise most real prescriptions, so the system must hold both generic and local brand names.

1.2 Problem Statement

There is no simple, typed, searchable record that links each doctor visit with its symptoms, diagnosis and prescribed medicines, and that automatically shows which medicines are antibiotics.

1.3 Objectives

- Store each doctor visit with date, doctor, symptoms and diagnosis.
- Store the medicines of each visit with dose, frequency and duration.
- Label antibiotics automatically on the server using a reference list of generic and Bangladeshi brand names.
- List, view, edit, delete and filter visits through a simple, responsive web interface.
- Build the system incrementally so that working software exists after every iteration.

1.4 Scope

In scope: typed data entry, MySQL persistence, server-side validation, antibiotic classification by name matching, Thymeleaf web interface.

Out of scope (prototype): authentication, REST APIs, OCR of prescriptions, AI-based medicine recognition, drug-interaction checking, and any form of medical advice. The antibiotic label is only a name match and is not a clinical decision.

1.5 Project Summary

Table 1: Project summary

Item	Detail
Project name	Shashtho Record
Internal name	`medtracker` (package and database name)
Type	Web application (server-rendered MVC)
Language and build	Java 17, Maven
Framework	Spring Boot (Web, Data JPA, Validation, Thymeleaf)
Database	MySQL, schema `medtracker`
Interface	Plain HTML, one CSS file, minimal inline JavaScript
Process model	Iterative and incremental
Planned duration	12 weeks, 5 iterations
Repository	[GitHub Repository Link]

2 Stakeholders, Existing Systems and Feasibility

2.1 Stakeholders

Table 2: Stakeholders

Stakeholder	Role	Interest
Patient / caregiver	Primary user	Keep a clear history of visits and medicines
Doctor (indirect)	Reads shared history	Quick view of past diagnoses and antibiotics
Developer	Builds and maintains	Clean, extensible design
Supervisor	Reviews proposal and progress	Meeting course requirements

2.2 Existing Approaches

Table 3: Comparison with existing approaches

Approach	Strength	Weakness	Shashtho Record
Paper prescriptions	No setup	Lost, unsearchable	Typed and searchable
Generic notes apps	Flexible	No structure, no antibiotic flag	Structured visit and medicine data
Hospital EMR	Complete	Tied to one hospital, not patient-owned	Patient-owned, local brand names

2.3 Feasibility Study

Table 4: Feasibility summary

Type	Assessment
Technical	Mature open-source stack; all needed features exist in Spring Boot and MySQL.
Economic	No licence cost. Development uses a personal computer and free tools.
Operational	Simple forms and tables; no training needed.
Schedule	Small scope fits 12 weeks with weekly increments.
Legal and ethical	Health data is sensitive. The prototype uses sample data only; encryption and login are future scope.

3 Requirements

3.1 Functional Requirements

Table 5: Functional requirements

ID	Requirement
FR1	Create a visit with date, doctor, symptoms, diagnosis and at least one named medicine.
FR2	Add and remove medicine rows dynamically on the form.
FR3	Edit an existing visit and its medicines.
FR4	Delete a visit after a confirmation prompt.
FR5	List visits newest first with doctor, diagnosis, medicine count and antibiotic count.
FR6	Filter the list to visits that contain antibiotics.
FR7	Show visit details with a medicine table and an “Antibiotic” badge.
FR8	Validate input on the server and show errors inline.
FR9	Set the antibiotic flag on the server only; ignore any client-supplied value.
FR10	Seed the antibiotic reference list at first startup if it is empty.
FR11	Show a 404 page for unknown visit ids.

3.2 Non-Functional Requirements

Table 6: Non-functional requirements

ID	Quality	Requirement
NFR1	Usability	Clean, readable, responsive layout; works on phone and desktop.
NFR2	Performance	List page loads in under one second for up to 1000 visits.
NFR3	Reliability	Visit and medicines are saved in one transaction.
NFR4	Maintainability	Layered packages: controller, service, repository, model, config.
NFR5	Portability	Runs on any OS with Java 17 and MySQL.
NFR6	Data integrity	Antibiotic reference names are unique and lowercase.
NFR7	Security (prototype)	JPA parameterised queries; Thymeleaf output escaping. No login in this version.

3.3 Assumptions and Constraints

- Single user on a local machine; no concurrent multi-user access.
- Antibiotic detection is name matching only (name equals or contains a reference name).
- The reference list covers common antibiotics and is not exhaustive.
- MySQL 8 is installed and running on port 3306.

4 Software Development Model: Iterative and Incremental

4.1 Overview

The project follows the iterative and incremental model. The system is built in small increments. Each iteration repeats the same cycle (plan, design, build, test, review) and ends with working software that adds to the previous increment. Feedback from each review shapes the next iteration.

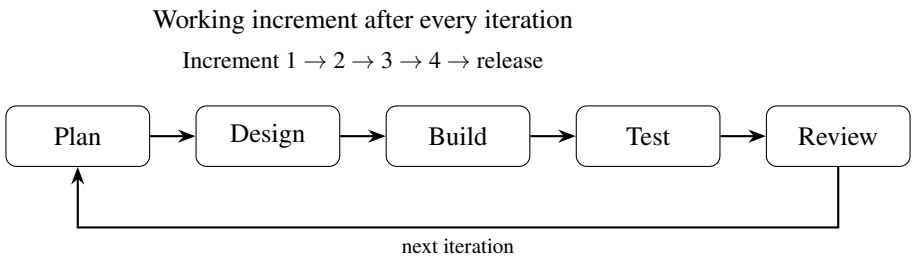


Figure 1: Iterative and incremental cycle used in Shashtho Record

4.2 Why This Model

Table 7: Process model comparison

Criterion	Waterfall	Iterative and incremental	Fit
Requirement changes	Costly	Absorbed each iteration	Medicine rules will evolve
Working software	At the end	After every iteration	Early demo to supervisor
Risk handling	Found late	Found early	Database and UI risks tested early
Documentation	Heavy upfront	Grows with product	Proposal first, refine later

5 Project Schedule, Milestones and Deliverables

5.1 Iteration Plan

Table 8: Iteration plan (12 weeks)

Iteration	Weeks	Goals	Increment delivered
0 – Inception	1–2	Requirements, feasibility, proposal, UML design	Proposal version 1
1 – Core CRUD	3–5	Entities, repositories, MySQL, list, add and detail pages	Visits with medicines can be stored and viewed
2 – Classification	6–8	Seeder, normalisation and matching, badge, edit, delete, filter	Antibiotics labelled automatically
3 – Refinement	9–10	Validation, 404 page, responsive CSS, testing, feedback fixes	Feature-complete prototype
4 – Release	11–12	Final testing, README, report, GitHub release	Final submission

5.2 Milestones

Table 9: Milestones

ID	Week	Milestone	Completion criteria
M1	2	Proposal approved	Proposal and UML diagrams reviewed
M2	5	Increment 1	Add, list and detail pages work with MySQL
M3	8	Increment 2	Antibiotic badge, edit, delete and filter work
M4	10	Increment 3	All functional requirements pass tests
M5	12	Final submission	Report, code and README on GitHub

5.3 Deliverables

Table 10: Deliverables

ID	Deliverable	Description	Due
D1	Project proposal	This document with UML diagrams	Week 2
D2	Increment 1	Source code, core CRUD	Week 5
D3	Increment 2	Source code, classification	Week 8
D4	Increment 3	Validated, responsive prototype	Week 10
D5	Test results	Manual test case table with results	Week 11
D6	Final package	Source, README, final report, GitHub link	Week 12

5.4 Resources and Effort

Table 11: Resources and effort estimate

Resource	Detail
Hardware	Personal computer
Software	JDK 17, Maven, MySQL 8, MySQL Workbench, IntelliJ IDEA, Git, LaTeX (all free)
Effort	About 10 hours per week for 12 weeks, about 120 hours
Cost	No monetary cost

5.5 Gantt Chart

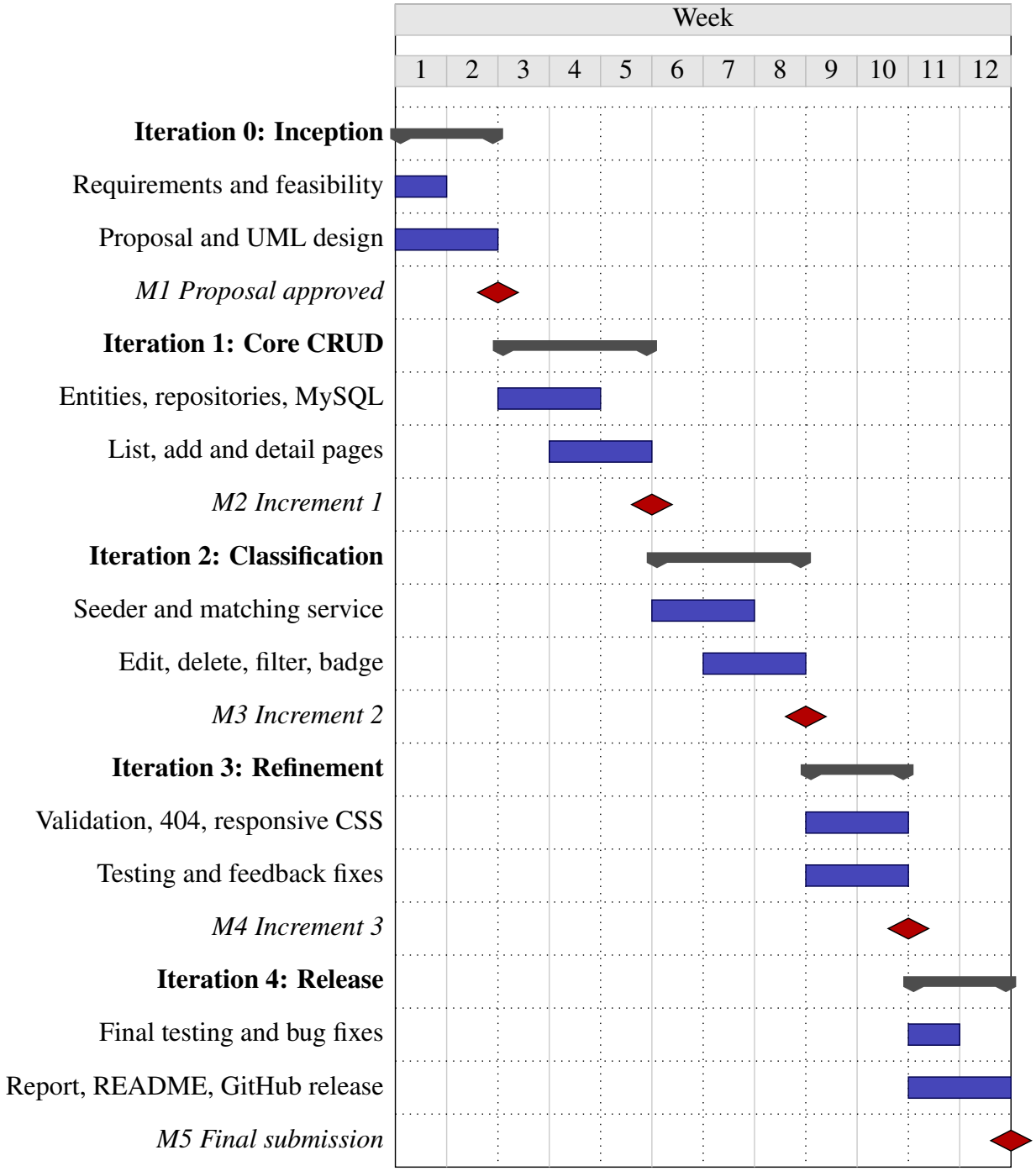


Figure 2: Project Gantt chart

6 UML and Design Diagrams

6.1 Data Flow Diagrams

Figure 3 shows the context diagram (Level 0). Figure 4 decomposes the system into three processes and two data stores (Level 1).

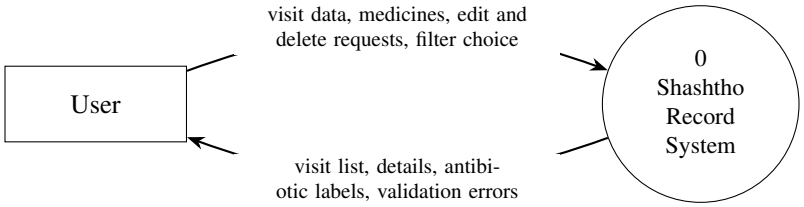


Figure 3: Data flow diagram, Level 0 (context)

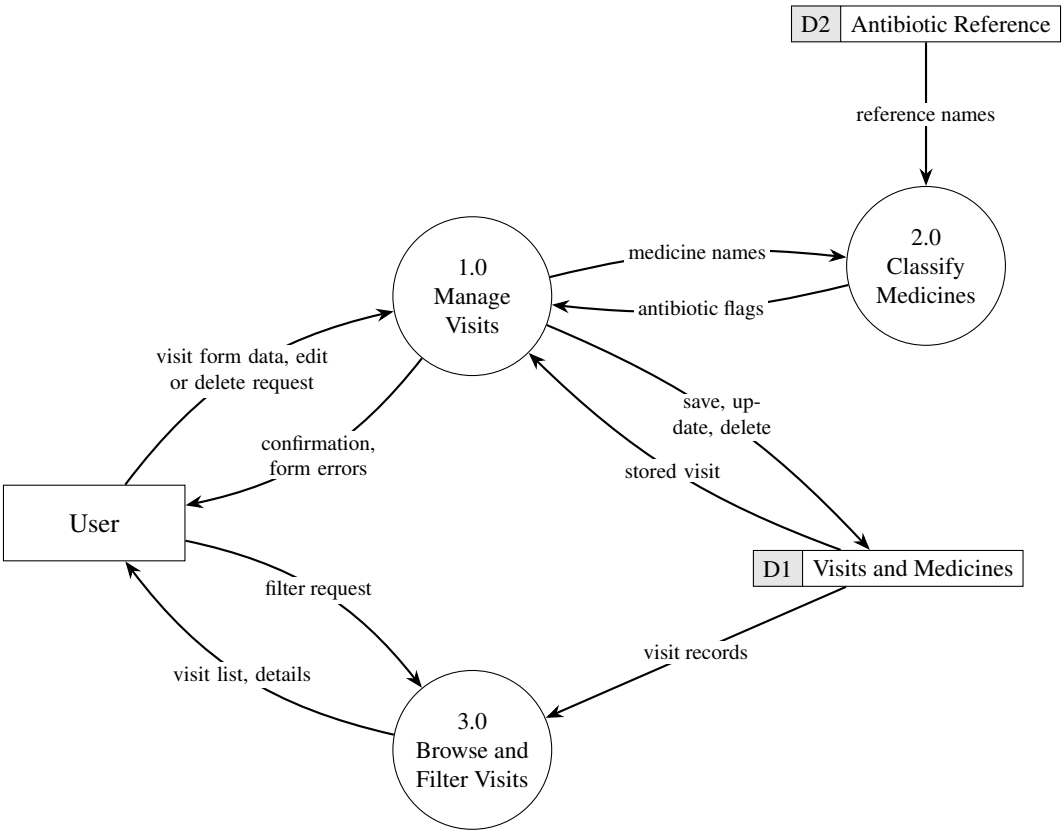


Figure 4: Data flow diagram, Level 1

6.2 Use Case Diagram

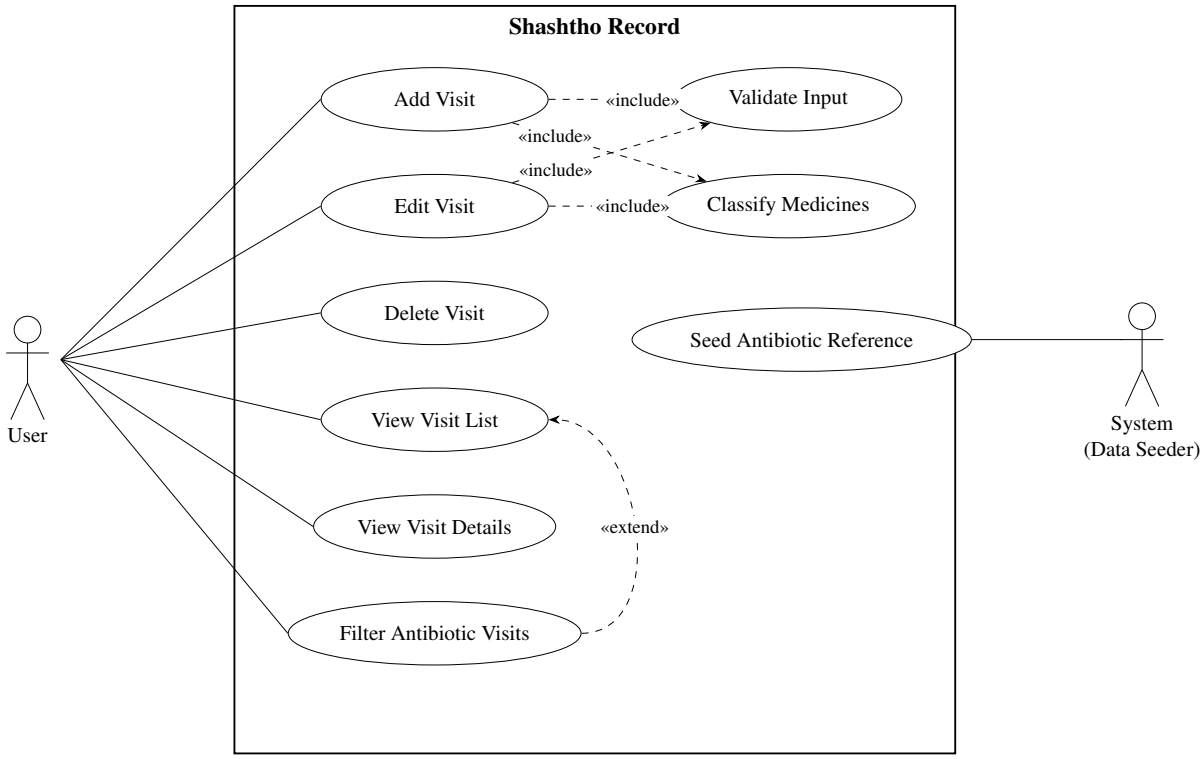


Figure 5: Use case diagram

Table 12: Use case summary

Use case	Actor	Description
Add Visit	User	Enter visit details and medicine rows, then save.
Edit Visit	User	Change details or medicines of a saved visit.
Delete Visit	User	Remove a visit after confirmation.
View Visit List	User	See visits newest first with counts.
View Visit Details	User	See one visit with its medicine table and badges.
Filter Antibiotic Visits	User	Show only visits that contain antibiotics.
Validate Input	System	Check date, doctor and at least one medicine name.
Classify Medicines	System	Set the antibiotic flag from the reference list.
Seed Antibiotic Reference	Data Seeder	Fill the reference table at first startup.

6.3 Activity Diagram

Figure 6 shows the activity flow for adding a visit.

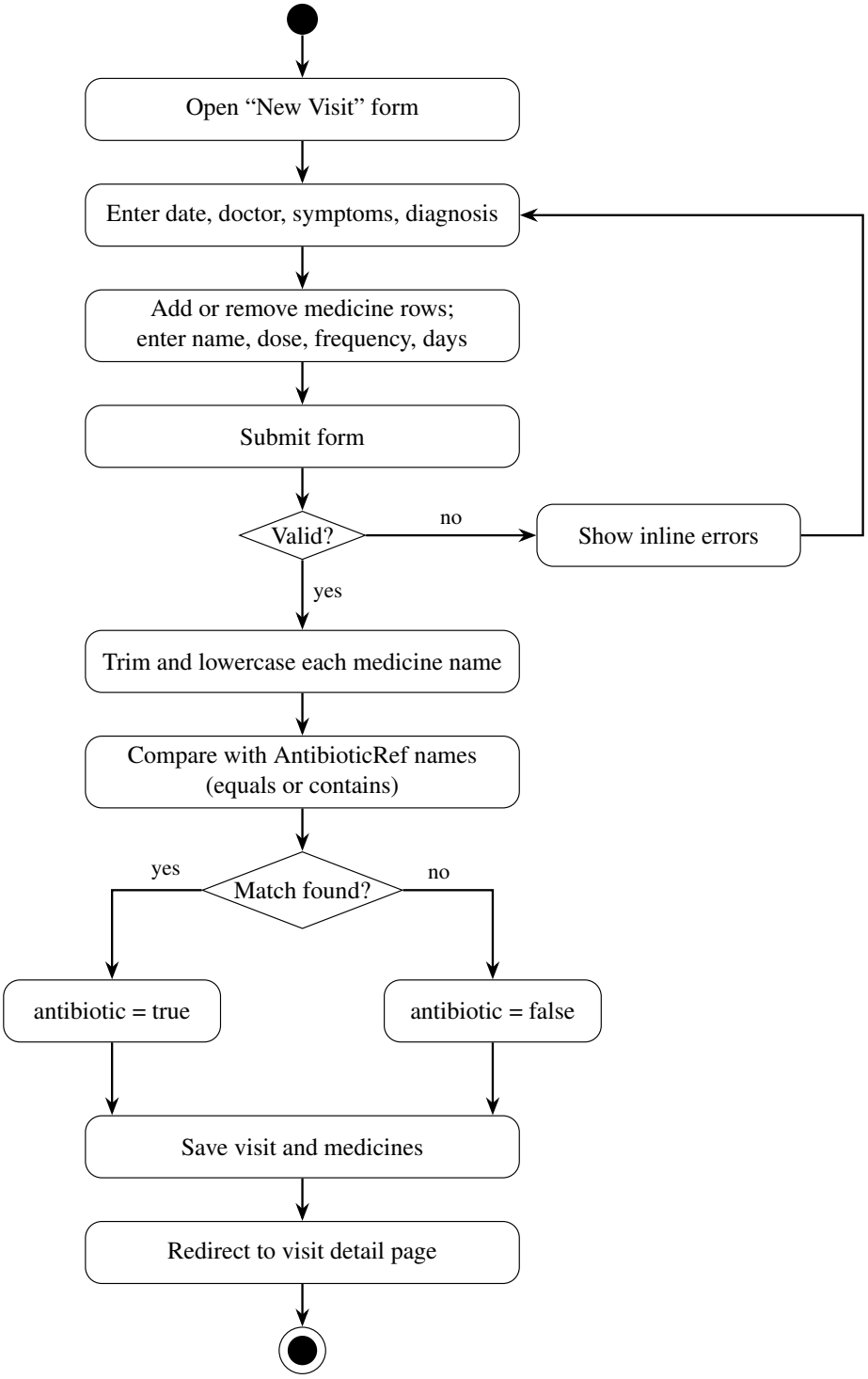


Figure 6: Activity diagram: add a visit

6.4 Entity Relationship Diagram

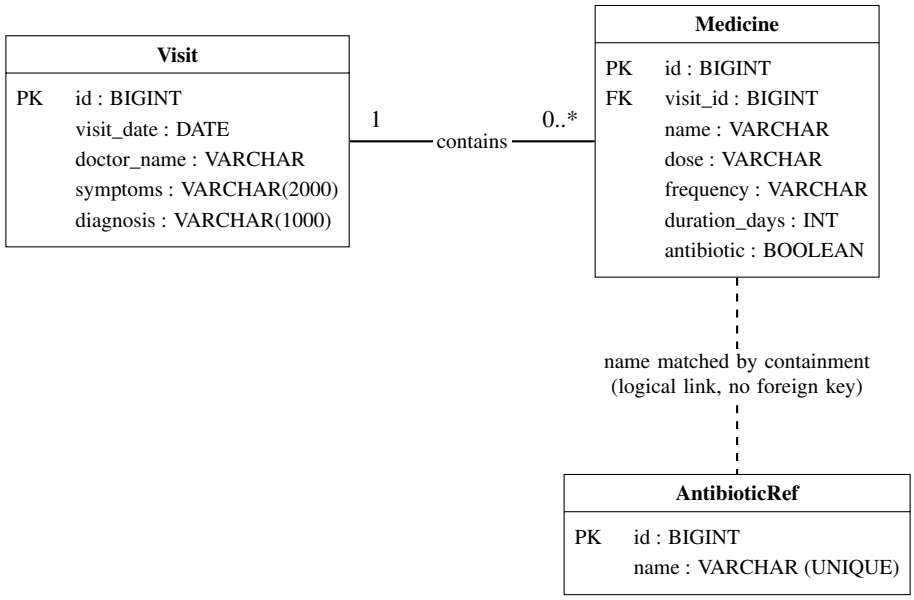


Figure 7: Entity relationship diagram

6.5 Class Diagram

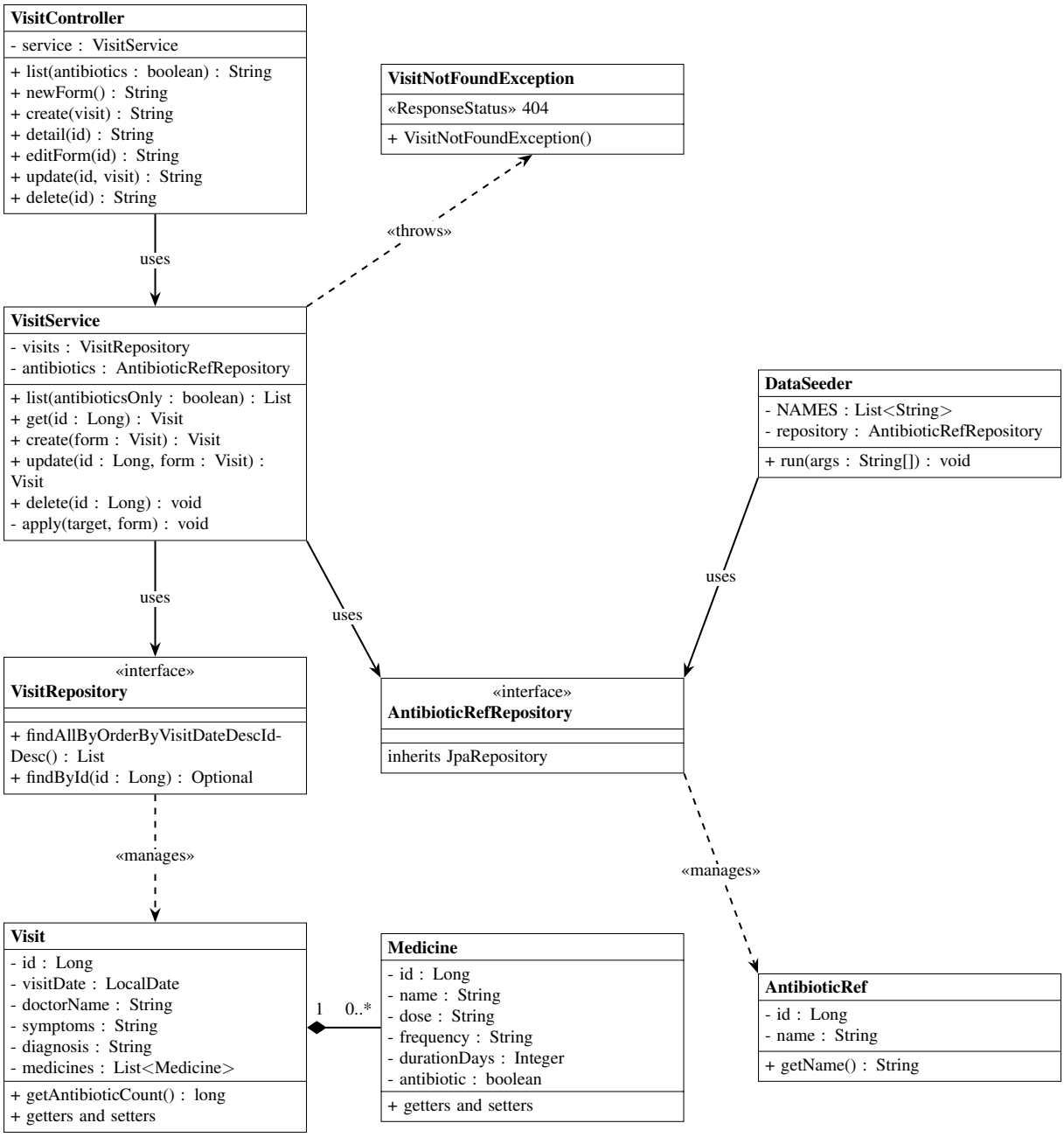


Figure 8: Class diagram

6.6 Object Diagram

Figure 9 shows one visit with two medicines at run time. One medicine matches an antibiotic reference name; the other does not.

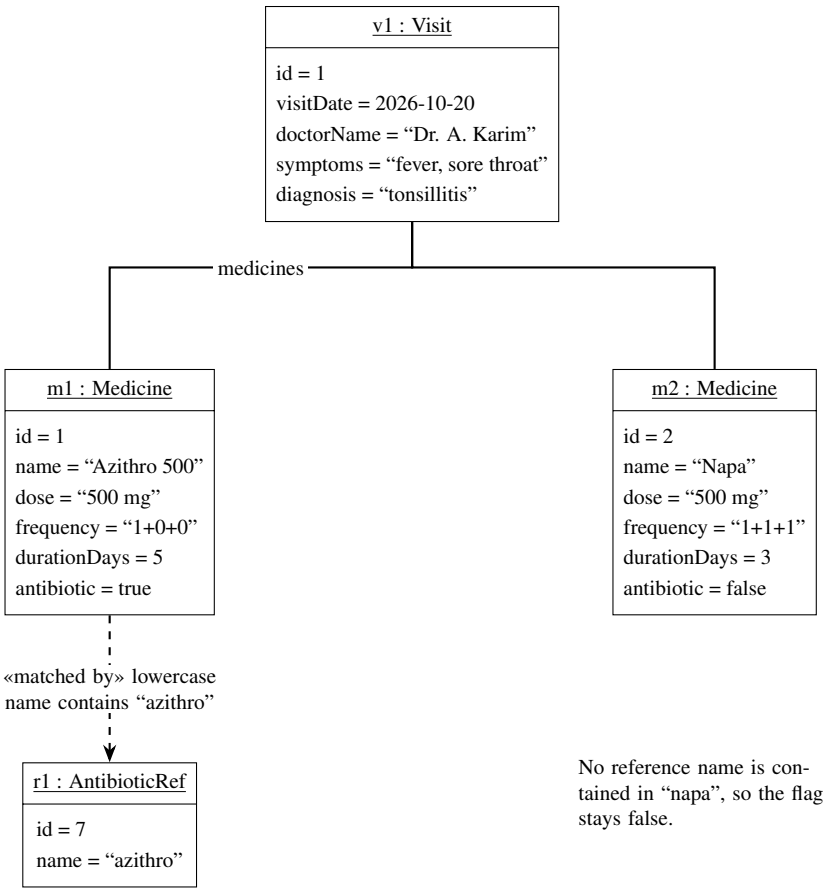


Figure 9: Object diagram

7 Implementation

7.1 Architecture

Shashtho Record is a layered Spring Boot MVC application. The controller handles HTTP requests and renders Thymeleaf templates. The service holds business rules and transactions. Repositories access MySQL through Spring Data JPA.

Table 13: Packages and classes

Package	Class	Responsibility
controller	VisitController	Routes, form binding, redirect after post
service	VisitService	CRUD, normalisation, antibiotic classification
service	VisitNotFoundException	Triggers the 404 page
repository	VisitRepository	Visit persistence with medicines fetched
repository	AntibioticRefRepository	Antibiotic reference persistence
model	Visit	Visit entity, validation rules
model	Medicine	Medicine entity, antibiotic flag
model	AntibioticRef	Reference name entity (unique, lowercase)
config	DataSeeder	Seeds reference names when the table is empty

The interface uses all nine classes listed above, which exceeds the minimum of five classes from the class diagram.

7.2 Classification Rule

The server ignores any client-supplied antibiotic value. For every medicine row it trims and lowercases the name and sets the flag when the name contains any reference name.

```
String name = m.getName().trim();
String key = name.toLowerCase(Locale.ROOT);
medicine.setAntibiotic(refs.stream().anyMatch(key::contains));
```

7.3 Screens

Table 14: Pages

Route	Method	Purpose
`/`	GET	Visit list, newest first, with antibiotics-only filter
`/visits/new`	GET	Empty visit form with dynamic medicine rows
`/visits`	POST	Create visit, then redirect
`/visits/{id}`	GET	Visit details with medicine table and badges
`/visits/{id}/edit`	GET	Edit form
`/visits/{id}`	POST	Update visit, then redirect
`/visits/{id}/delete`	POST	Delete visit after a confirm prompt

8 Testing Plan

Table 15: Manual test cases

ID	Req.	Test	Expected result
T1	FR1	Save visit with date, doctor and one medicine	Visit saved, detail page shown
T2	FR8	Submit with empty doctor	Inline error under doctor field
T3	FR8	Submit with all medicine names blank	Inline error on medicines
T4	FR2	Click add and remove medicine row	Rows appear and disappear; at least one row remains
T5	FR9	Enter “Zimax 500”	Antibiotic badge shown
T6	FR9	Enter “Napa”	No badge
T7	FR9	Tamper the antibiotic field in the request	Value ignored; server result used
T8	FR3	Edit a visit, remove one medicine	Medicine removed from database
T9	FR4	Delete and cancel, then delete and confirm	Cancel keeps; confirm removes visit
T10	FR6	Tick “Antibiotics only”	Only visits with antibiotics listed
T11	FR10	Restart application	Reference list not duplicated
T12	FR11	Open `/visits/9999`	404 page

9 Risk Management

Table 16: Risk register

Risk	Prob.	Impact	Mitigation
False antibiotic match from short names	Medium	Medium	Use specific names; review list each iteration
Missing brand names in reference list	High	Medium	Seed list is editable data, extended per iteration
MySQL setup problems	Medium	High	Documented setup; test database in Iteration 1
Schedule slip	Medium	Medium	Fixed scope per increment; drop optional polish first
Sensitive health data exposure	Low	High	Sample data only; login and encryption planned

10 Future Scope

- OCR of prescriptions
- AI medicine recognition
- Interaction alerts
- Login and encryption
- Audit log

11 Conclusion

Shashtho Record gives a simple, structured record of doctor visits and flags antibiotics automatically using generic and local brand names. The iterative and incremental plan delivers a working increment every few weeks, keeps risk low, and leaves a clear path to OCR, AI recognition and security features.

References

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2019.

[3] M. Fowler, *UML Distilled*, 3rd ed., Addison-Wesley, 2003.

[4] Spring Boot Reference Documentation, https://docs.spring.io/spring-boot/.

[5] Thymeleaf Documentation, https://www.thymeleaf.org/documentation.html.

[6] MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/.